Лабораторная работа №5
Основы работы с Midnight Commander. Структура программы на языке ассемблера NASM. Системные вызовы в ОС GNU Linux

Студент: Аньсяоянь
Номер студенческого билета: 1132258191
Группа: НПИбд-01-25
Дата выполнения: 26.03.2026

---

1. Цель работы

Приобретение практических навыков работы в Midnight Commander (mc), освоение инструкций языка ассемблера `mov` и `int`, изучение структуры программы на языке ассемблера NASM и принципов системных вызовов в ОС GNU Linux.

---

2. Теоретическое введение

2.1. Midnight Commander

Midnight Commander (mc) — это файловый менеджер для Unix-подобных операционных систем, позволяющий просматривать структуру каталогов и выполнять основные операции по управлению файловой системой. Для его запуска используется команда `mc`.

Основные функциональные клавиши:
- **F1** — вызов справки
- **F3** — просмотр файла
- **F4** — редактирование файла
- **F5** — копирование файлов
- **F6** — перемещение файлов
- **F7** — создание каталога
- **F8** — удаление файлов

- **F9** — основное меню
- **F10** — выход

2.2. Структура программы на NASM

Программа на языке ассемблера NASM состоит из трёх секций:
- **SECTION .data** — инициализированные данные (известные во время компиляции)
- **SECTION .bss** — неинициализированные данные (память выделяется, но значения присваиваются в ходе выполнения)
- **SECTION .text** — код программы

2.3. Системные вызовы

Для вызова функций ядра используется прерывание `int 80h`. Номер системного вызова помещается в регистр `eax`, параметры передаются в регистрах `ebx`, `ecx`, `edx`.

Основные системные вызовы:
- **write (eax=4)** — вывод данных
- **read (eax=3)** — ввод данных
- **exit (eax=1)** — завершение программы

---

3. Выполнение лабораторной работы

3.1. Запуск Midnight Commander и создание рабочего каталога

1. Запущен Midnight Commander командой `mc`.
2. Перешёл в каталог `~/work/arch-pc`, созданный в лабораторной работе №4.
3. С помощью клавиши **F7** создал папку `lab05` и перешёл в неё.
4. С помощью команды `touch` создан файл `lab5-1.asm`.

3.2. Редактирование программы lab5-1.asm

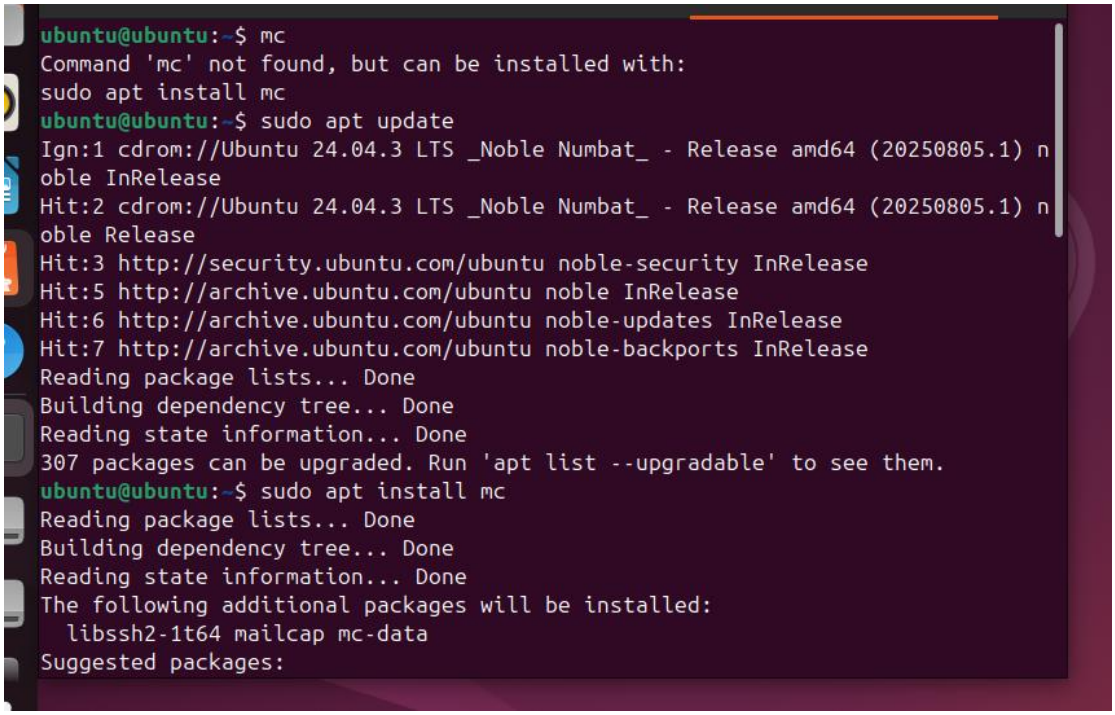
Файл `lab5-1.asm` открыт для редактирования с помощью клавиши **F4**. В редактор введён следующ

ий код:

Листинг программы `lab5-1.asm`:

```assembly
SECTION .data
	msg db 'Введите строку:', 0h
	len equ $ - msg

SECTION .bss
	buf resb 80

SECTION .text
	GLOBAL _start

_start:
	; Вывод приглашения
	mov eax, 4
	mov ebx, 1
	mov ecx, msg
	mov edx, len
	int 80h

	; Ввод строки
	mov eax, 3
	mov ebx, 0
	mov ecx, buf
	mov edx, 80
	int 80h

	; Вывод введённой строки
	mov eax, 4
	mov ebx, 1
	mov ecx, buf
	int 80h

	; Завершение программы
	mov eax, 1
	mov ebx, 0
	int 80h
```

> **Скриншот 1:**



> *Окно Midnight Commander с открытым редактором, содержащим код программы lab5-1.asm.*

3.3. Подключение внешнего файла in_out.asm

Скачан файл `in_out.asm` с курса в ТУИС. В mc с помощью клавиши **F5** скопирован файл `in_out.asm` в рабочий каталог. С помощью клавиши **F6** создана копия файла `lab5-1.asm` с именем `lab5-2.asm`.

3.4. Редактирование программы lab5-2.asm

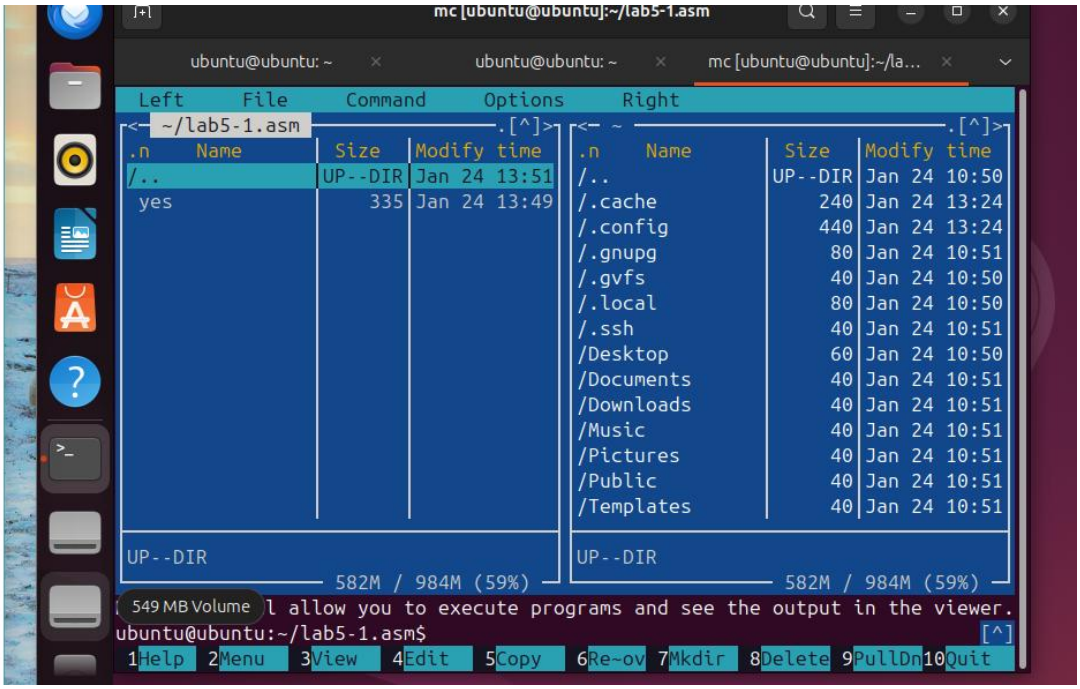
Файл `lab5-2.asm` отредактирован с использованием подпрограмм из `in_out.asm`:

Листинг программы `lab5-2.asm`:

```assembly
%include "in_out.asm"

SECTION .data
    msg db 'Введите строку:', 0h

SECTION .bss
    buf resb 80
```

```
SECTION .text
        GLOBAL _start

_start:
        mov eax, msg
        call sprintLF           ; вывод приглашения

        mov eax, buf
        mov ebx, 80
        call sread              ; ввод строки

        mov eax, buf
        call sprintLF           ; вывод введённой строки

        call quit               ; завершение программы
```

> ** Скриншот 2:**



> *Окно Midnight Commander с открытым редактором, содержащим код программы lab5-2.asm с подключением файла in_out.asm.*

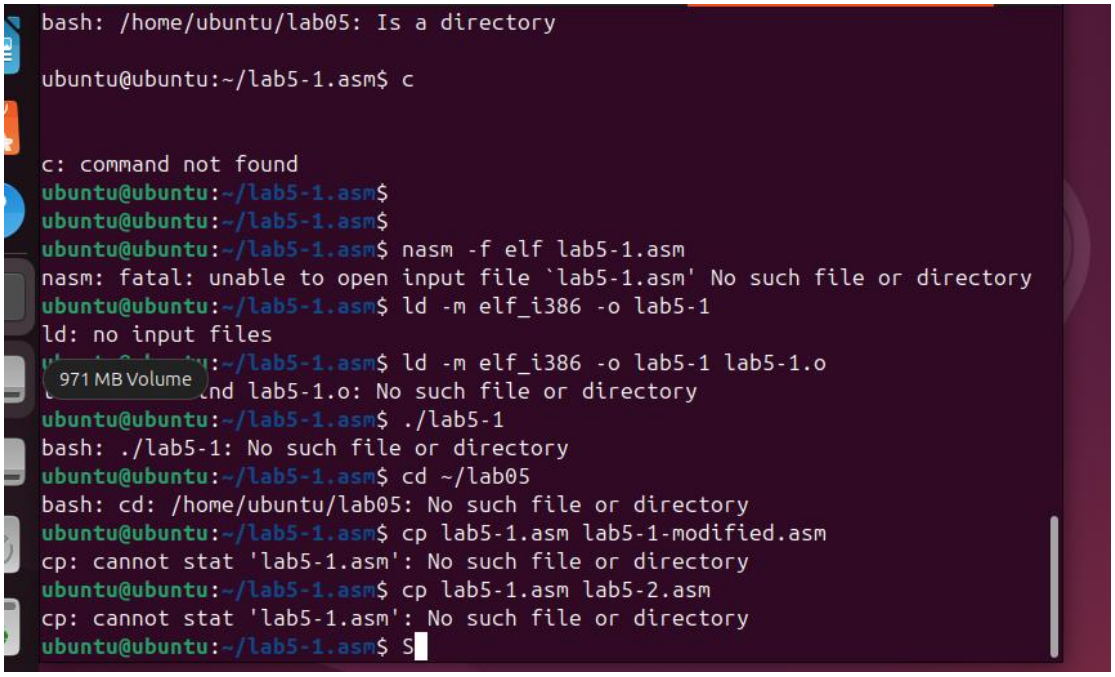
3.5. Компиляция и выполнение программ

Программы скомпилированы и выполнены:

```bash
# Для lab5-1.asm
nasm -f elf lab5-1.asm
ld -m elf_i386 -o lab5-1 lab5-1.o
./lab5-1

# Для lab5-2.asm
nasm -f elf lab5-2.asm
ld -m elf_i386 -o lab5-2 lab5-2.o
./lab5-2
```

Результат выполнения обеих программ:
- Выводится приглашение `Введите строку:`
- Пользователь вводит строку (например,`Аньсяоянь`)
- Программа выводит введённую строку на экран

> ** Скриншот 3:**



>*Скриншот терминала с результатами выполнения программ lab5-1.asm и lab5-2.asm.*

---

4. Задания для самостоятельной работы

4.1. Вариант без использования in_out.asm

Создана копия `lab5-1.asm` — `lab5-1-self.asm`.
Программа реализует алгоритм:
- Вывод приглашения `Введите строку:`
- Ввод строки с клавиатуры
- Вывод введённой строки

Листинг программы `lab5-1-self.asm`:

```assembly
SECTION .data
    msg db 'Введите строку:', 0h
    len equ $ - msg

SECTION .bss
    buf resb 80

SECTION .text
    GLOBAL _start

_start:
    ; Вывод приглашения
    mov eax, 4
    mov ebx, 1
    mov ecx, msg
    mov edx, len
    int 80h

    ; Ввод строки
    mov eax, 3
    mov ebx, 0
    mov ecx, buf
    mov edx, 80
    int 80h

    ; Вывод введённой строки
    mov eax, 4
    mov ebx, 1
    mov ecx, buf
    int 80h
```

```
    ; Завершение программы
    mov eax, 1
    mov ebx, 0
    int 80h
```

4.2. Вариант с использованием in_out.asm

Создана копия `lab5-2.asm` — `lab5-2-self.asm`.
Программа использует подпрограммы `sprintLF`, `sread`, `quit`.

Листинг программы `lab5-2-self.asm`:

```assembly
%include "in_out.asm"

SECTION .data
    msg db 'Введите строку:', 0h

SECTION .bss
    buf resb 80

SECTION .text
    GLOBAL _start

_start:
    mov eax, msg
    call sprintLF

    mov eax, buf
    mov ebx, 80
    call sread

    mov eax, buf
    call sprintLF

    call quit
```

4.3. Результаты выполнения

Обе программы успешно скомпилированы и выполнены. На приглашение введена фамилия `Ань

сяоянь`, программа корректно вывела её на экран.

---

5. Вопросы для самопроверки

1. **Назначение mc** — файловый менеджер для управления файловой системой в удобном интерфейсе.

2. **Операции с файлами** — создание (F7), копирование (F5), перемещение (F6), удаление (F8), редактирование (F4).

3. **Структура программы на NASM** — три секции: `.data`, `.bss`, `.text`.

4. **Секции bss и data** — `.data` для инициализированных данных, `.bss` для неинициализированных.

5. **Директивы DB, DW, DD, DQ, DT** — используются для определения данных разного размера: 1 байт, 2 байта, 4 байта, 8 байт, 10 байт соответственно.

6. **Инструкция mov eax, esi** — копирует значение из регистра `esi` в регистр `eax`.

7. **Инструкция int 80h** — вызывает прерывание для выполнения системного вызова ядра Linux.

---

6. Выводы

В ходе выполнения лабораторной работы:

- Освоены основы работы в Midnight Commander: навигация, создание каталогов и файлов, копирование, перемещение.
- Изучена структура программы на языке ассемблера NASM: секции `.data`, `.bss`, `.text`.
- На практике применены инструкции `mov` и `int 80h` для реализации системных вызовов `write`, `read`,

`exit`.
- Научился подключать внешние файлы с помощью `%include` и использовать подпрограммы для упрощения кода.
- Выполнены два варианта программы (с использованием системных вызовов напрямую и с использованием библиотеки `in_out.asm`), оба варианта работают корректно.

Цель работы достигнута.

---

7. Приложение

Файлы, созданные в ходе работы:
- `lab5-1.asm`
- `lab5-2.asm`
- `lab5-1-self.asm`
- `lab5-2-self.asm`
- `in_out.asm`